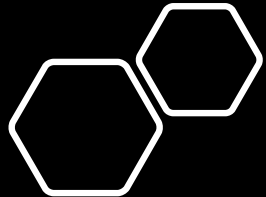


# Chapter 6. Pointers



# Nội dung

Một số khái niệm và phép toán trên con trỏ

Con trỏ với mảng

Con trỏ với struct

Con trỏ với danh sách liên kết

Con trỏ với cấu trúc cây

# Một số khái niệm và phép toán trên con trỏ

Địa chỉ ô nhớ: Mỗi biến chiếm một số ô nhớ, địa chỉ ô nhớ được biểu thị ở hệ đếm 16

Con trỏ: là một biến, trỏ đến địa chỉ của một ô nhớ

Khai báo con trỏ: <kiểu dữ liệu> \*p

Phép lấy địa chỉ của một biến con trỏ p: &p

Phép toán lấy giá trị tại một địa chỉ trỏ bởi p: \*p

Nếu p là con trỏ trỏ tới biến x thì các cách viết x và \*p là tương đương (đồng nhất) nhau.

Cấp phát động bộ nhớ: new;

Giải phóng khối nhớ đã được cấp phát: delete

## Con trỏ với mảng

- Con trỏ với mảng 1 chiều

Các cách viết sau đây là tương đương:

$$*(a+i) \Leftrightarrow a[i];$$

$$a \Leftrightarrow \&a[0];$$

$$a+i \Leftrightarrow \&a[i];$$

- Con trỏ với mảng 2 chiều

Phần tử ở dòng  $i$  cột  $j$  của mảng  $a$  có  $m$  dòng,  $n$  cột được truy xuất bởi  $*(a+n*i+j)$ .

## Ví dụ 1

Hãy cho biết kết quả của chương trình sau:

```
1. #include <iostream.h>
2. int main()
3. {
4.     int ar[]={12,10,11,18,13,6},*p;
5.     p=&ar[0];
6.     printf("%d ",*p);
7.     p=&ar[3];
8.     printf("%d ",*p);
9.     *p=*p+100;
10.    printf("%d ",ar[3]);
11.    p=p-2;
12.    printf("%d ",*p);
13.}
```

## Ví dụ 2 (con trỏ với mảng 1 chiều)

Cho dãy  $n$  số nguyên. Hãy thực hiện các công việc sau:

a. Tìm giá trị lớn nhất của dãy.

b. Sắp xếp các phần tử của dãy theo thứ tự tăng dần.

```
1.  #include <alloc.h>
2.  #include <iostream.h>
3.  void nhap(int *&a, int &n);
4.  void xuat(int *a, int n);
5.  int giatrilonnhat(int *a, int n);
6.  void swap(int *a, int *b);
7.  void sapxep(int *a, int n);
8.  int main()
9.  {int *a,n;
10.   nhap(a,n);
11.   cout<<giatrilonnhat(a,n)<<endl;
12.   sapxep(a,n);
13.   xuat(a,n);
14.   delete a;
15. }
```

```
16. void nhap(int *&a, int &n)
17. {
18.     cout<<"Nhap so phan tu cua mang :";cin>>n;
19.     a=new int [n];
20.     for (int i=0;i<n;i++)
21.         cin>>*(a+i);
22. }
23. void xuat(int *a, int n)
24. {
25.     for (int i=0;i<n;i++)
26.         cout<<*(a+i)<<" ";
27. }
```

```
28. int giatrilonnhhat(int *a, int n)
29. {
30.     int max=*a;
31.     for (int i=1;i<n;i++)
32.         if (*(a+i)>max)
33.             max=*(a+i);
34.     return max;
35. }
```

```
36. void swap(int *a, int *b)
37. {
38.     int t=*a;
39.     *a=*b;
40.     *b=t;
41. }
42. void sapxep(int *a, int n)
43. {
44.     for (int i=0;i<n-1;i++)
45.         for (int j=i+1;j<n;j++)
46.             if (*(a+i)>*(a+j))
47.                 swap(a+i,a+j);
48. }
```

- Note: hàm swap có thể thiết kế như sau:  
void swap(int &a, int &b)

## Ví dụ 3 (con trỏ với mảng 2 chiều)

Viết chương trình nhập mảng hai chiều  $a$  có  $m$  dòng  $n$  cột, các phần tử là các số nguyên và một số nguyên  $x$ .

a. Hãy đếm xem trong mảng có bao nhiêu số bằng  $x$ .

b. Cho biết vị trí của các phần tử bằng  $x$ .

```
1.  #include<iostream.h>
2.  #include<alloc.h>
3.  void  nhap(int *&a,int &m, int &n, int &x);
4.  void  demsobangx(int *a, int m, int n, int x);
5.  int   main()
6.  {
7.      int *a,m,n,x;
8.      / nhap(a,m,n);
9.      demsobangx(a,m,n,x);
10.   delete a;
11. }
```

```
12. void nhap(int *&a,int &m, int &n, int &x);  
13. {  
14.     cout<<"Nhap vao so dong:";     cin>>m;  
15.     cout<<"Nhap vao so cot:";     cin>>n;  
16.     a=new int [m*n];  
17.     for (int i=0;i<m;i++ )  
18.         for (int j=0;j<n;j++ )  
19.             cin>>*(a+n*i+j);  
20.     cout<<"Nhap vao gia tri x:";     cin>>x;  
21. }
```

```
22. void demsobangx(int *a, int m, int n, int x)
23. {
24.     int d=0;
25.     for (int i=0;i<m;i++)
26.         for (int j=0;j<n;j++)
27.             if (x==*(a+n*i+j))
28.                 {
29.                     d++;
30.                     cout<<i<<" "<<j<<endl;
31.                 }
32.     cout<<"So lan xuat hien cua x la : "<<d;
33. }
```

## Ví dụ 4 (con trỏ với struct)

Viết chương trình thực hiện các công việc sau:

- a. Nhập vào hồ sơ của n nhân viên với các thông tin: họ và tên, ngày sinh, lương cơ bản, thưởng, thực lãnh; trong đó thực lãnh = lương cơ bản + thưởng.
- b. In danh sách nhân viên theo **thực lãnh** giảm dần.

# Hãy cho biết kết quả của các chương trình sau (CFREE 5.0)

a.

```
#include <iostream.h> //CFREE 5.0
int main()
{
    int a[]={4,3,7,9,2,6},*p;
    p= new int;
    p=&a[0]; cout<<*p<<endl;
    p=&a[4]; *p=*p+3; cout<<a[4]<<endl;
    p=p-3; cout<<*p<<endl;
    p=p+4; cout<<*p<<endl;
    delete p;
    return 0;
}
```

b.

```
#include <iostream.h> //CFREE 5.0
int main()
{
    int a[]={20,11,2020,26,3,2020},*p;
    p= new int;
    p=&a[0];
    cout<<*p<<endl;
    p=&a[4];
    *p=*p+3;
    cout<<a[4]<<endl;
    p=p-3;
    cout<<*p<<endl;
    p=p+4;
    cout<<*p<<endl;
    delete p;
    return 0;
}
```

C.

```
#include <iostream.h>
int main()
{
    int a[]={15,2021,6,14,30,3},*p;
    p= new int;
    p=&a[1];
    cout<<*p<<endl;
    p=&a[4];
    *p=*p+3;
    cout<<a[4]<<endl;
    cout<<*(p-2)<<endl;
    cout<<*(p+1)<<endl;
    delete p;
    return 0;
}
```

d.

```
#include<iostream.h>
void swap(int *a,int *b)
{
    int t=*a;*a=*b;*b=t;
}
int main()
{
    int a=18,b=5;
    swap(&a,&b);
    cout<<a<<" "<<b;
    return 0;
}
```

e.

```
#include<iostream.h>
int main()
{
    int *a,m=4,n=5;
    a=new int[m*n];
    for (int i=0;i<m;i++)
    for (int j=0;j<n;j++)
        *(a+n*i+j)=i*j+1;
    cout<<*(a+13);
    delete a;
    return 0;
}
```

f.

```
#include<iostream.h>
int main()
{
    int *p;
    int a=2021;
    p=NULL;
    cout<<p+a<<endl;
    return 0;
}
```

g.

```
#include<iostream.h>
int *test(int a);
int main()
{   int a=12;
    //cout<<test(a)<<endl;
    //Gia su ket qua dong tren la 0x66ff00
    cout<<test(a)+14;
    return 0;
}
int *test(int a)
{
    int *p;
    p=&a;
    return p;
}
```

h.

```
#include<iostream.h>
int main()
{
    int *a,N=24;
    a=new int[N];
    for (int i=0;i<N;i++)
        *(a+i)=2*i+1;
    int n=6,i=2,j=3;
    cout<<*(a+n*i+j);
    delete a;
    return 0;
}
```

# Con trỏ với danh sách liên kết

(Hướng dẫn sinh viên nghiên cứu)

# Con trỏ với danh sách liên kết

Với cách tổ chức dữ liệu như mảng một chiều hoặc mảng hai chiều thì mỗi liên kết giữa các phần tử là một đặc trưng quan trọng của cấu trúc dữ liệu mảng; nó thuận tiện cho nhiều bài toán trong thực tế; nhưng cũng bất lợi trong việc xử lý nhiều bài toán khác; chẳng hạn như các bài toán chèn/xóa phần tử trong dãy,...

# Tổ chức dữ liệu dạng danh sách (...)

Mục này trình bày cách tổ chức dữ liệu linh hoạt hơn, đó là tổ chức dữ liệu dạng danh sách liên kết;

- Mỗi phần tử của danh sách được trỏ bởi một con trỏ; các khối nhớ được cấp cho một danh sách không nhất thiết phải liên tục nhau.
- Một phần tử của danh sách có hai thành phần:
  - oPhần thứ nhất lưu trữ các thông tin về bản thân phần tử,
  - oPhần thứ hai lưu trữ địa chỉ của phần tử kế tiếp trong danh sách, hoặc lưu trữ giá trị NULL nếu là phần tử cuối danh sách.

# Cấu trúc dữ liệu một phần tử trong danh sách liên kết đơn

```
struct node
{
    data      info;//data là kiểu dữ liệu đã định nghĩa trước
    node*     next;//con trỏ chỉ đến cấu trúc node
};
```

Mỗi phần tử của danh sách đơn chứa hai thông tin :

- Thành phần dữ liệu: lưu trữ các thông tin về bản thân phần tử .
- Thành phần nối liên kết: lưu trữ địa chỉ của phần tử kế tiếp trong danh sách, hoặc lưu trữ giá trị NULL nếu là phần tử cuối danh sách.

# Quản lý danh sách

- Để quản lý một xâu đơn chỉ cần biết địa chỉ phần tử đầu xâu.
- Thường một con trỏ head sẽ được dùng để lưu trữ địa chỉ phần tử đầu xâu, ta gọi head là đầu xâu. Ta có khai báo node \*head;
- Có thể sử dụng thêm một con trỏ tail giữ địa chỉ phần tử cuối xâu. Khai báo node \*tail;

# Các thao tác cơ bản trên danh sách đơn

Giả sử có các định nghĩa sau:

```
struct node
{
    int    info;
    node *next;
};

struct list
{
    node *head,*tail;
};

list    head,tail;
```

# Hàm *tạo nút* (getnode(x))

```
node*    getnode (int x)
{
    p=new node;
    if (p==NULL)
        exit(1);
    p->info=x;
    p->next=NULL;
    return p;
}
```

Để tạo ra một phần tử cho danh sách với thông tin chứa trong x

# Chèn một phần tử vào danh sách

Có 3 loại thao tác chèn một phần tử  $x$  được trả bởi con trỏ `new_element` vào danh sách:

- Chèn vào đầu danh sách
- Chèn vào cuối danh sách
- Chèn vào sau phần tử được trả bởi con trỏ `q`

# Chèn vào đầu danh sách

```
void insert_head(list &l, node *p)
{
    if (l.head==NULL)
    {
        l.head=p; // new_element như mô tả trên
        l.tail=p;
    }
    else
    {
        p->next=l.head;
        l.head=p;
    }
}
```

# Tạo danh sách bằng cách chèn đầu

```
void list_insert_head (list &l)
{
    int x,n;
    cout<<"Nhap vao so n = ";cin>>n;
    for (int i=1;i<=n;i++)
    {
        cin>>x;
        insert_head(l,getnode(x));
    }
}
```

# Chèn vào cuối danh sách

```
void    insert_tail(list &l,node *p)
{
    if (l.head==NULL)
        {
            l.head=p;
            l.tail=p;
        }
    else
        {
            l.tail->next=p;
            l.tail=p;
        }
}
```

# Tạo danh sách bằng cách chèn cuối

```
void list_insert_tail(list &l)
{
    int x,n;
    cout<<"Danh sach co bao nhieu phan tu : ";cin>>n;
    for (int i=1;i<=n;i++)
    {
        cin>>x;
        insert_tail(l,getnode(x));
    }
}
```

# Chèn vào danh sách sau một phần tử q

```
void insert_after_nodeq(list &l,node *q,int x)
{
    node *p=taonut(x);
    if (p==NULL) exit(1);
    if (q!=NULL)
    {
        p->next=q->next;
        q->next=p;
        if (q==l.tail)
            l.tail=p;
    }
    else
        insert_head(l,p);
}
```

# Tìm một phần tử trong danh sách đơn

```
node* search(list l,int x)
{
    p=l.head;
    while ((p!=NULL)&&(p->Info!=x))
        p=p->next;
    return p;
}
```

Note:

- Viết lại bằng cách sử dụng vòng lặp for
- Viết lại nếu giá trị trả về là 1/0 nếu tìm thấy hoặc không tìm thấy

# Duyệt danh sách

```
void list_process(list l)
{
    p=l.head;
    while (p!=NULL)
    {
        cout<<p->Info<<" ";
        p=p->next;
    }
    cout<<endl;
}
```

# Sắp xếp danh sách

- Phương án 1: Hoán vị nội dung các phần tử trong danh sách (thao tác trên vùng info).
- Phương án 2: Thay đổi các mối liên kết (thao tác trên vùng next)

# Phương án 1:

Cài đặt giải thuật sắp xếp Chọn trực tiếp trên xâu

```
void list_selection(list l)
{
    for (node *p=l.head;p!=l.tail;p=p->next)
    {
        node *min=p;
        for (node *q=p->next;q!=NULL;q=q->next)
            if (q->info<min->info)
                min=q;
        int temp=min->info;
        min->info=p->info;
        p->info=temp;
    }
}
```

# Cài đặt giải thuật sắp xếp đổi chỗ trực tiếp trên chuỗi

```
void list_interchangesort(list l)
{
    for (node *p=l.head;p!=l.tail;p=p->next)
        for (node *q=p->next;q!=NULL;q=q->next)
            if (p->info>q->info)
                {
                    int temp=p->info;
                    p->info=q->info;
                    q->info=temp;
                }
}
```

## Phương án 2:

Thay đổi các mối liên kết (thao tác trên vùng next)

- Bước1: Khởi tạo danh sách mới Result là rỗng;
- Bước2: Tìm trong danh sách cũ 1 phần tử nhỏ nhất;
- Bước3: Tách min khỏi danh sách l;
- Bước4: Chèn min vào cuối danh sách Result;
- Bước5: Lặp lại bước 2 khi chưa hết danh sách head;

## Hãy cho biết kết quả của chương trình sau:

```
#include<iostream.h>
struct node
{
int info;
node *next;
};
struct list
{
node *head,*tail;
};
```

```
int main()
{
node *p,*q,*r;
list l;
int a=10, b=20, c=30;
p=new node;
p->info=a;
l.head=p;
q=new node;
q->info=b;
r=new node;
r->info=c;
p->next=q;
q->next=r;
r->next=NULL;
```

```
l.tail=r;
cout<<l.head->info<<" ";
cout<<l.tail->info<<" ";
node *t=l.head;
while (t!=NULL)
{
cout<<t->info<<" ";
t=t->next;
}
delete p;
delete q;
delete r;
return 0;
}
```

# VÍ DỤ 5

Cho một danh sách liên kết đơn l, mỗi nút là một số nguyên

- a. Tạo danh sách l chứa n số nguyên.
- b. In danh sách
- c. Tìm xem x có thuộc về danh sách l hay không ? Nếu có trả về con trỏ trỏ đến nút tìm được; nếu không trả về giá trị NULL
- d. Tìm xem x có thuộc về danh sách l hay không ? Nếu có trả về giá trị 1; nếu không trả về giá trị 0.
- e. Sắp xếp các phần tử theo chiều tăng dần bằng phương pháp đổi chỗ trực tiếp
- f. Sắp xếp các phần tử theo chiều tăng dần bằng phương pháp chọn trực tiếp
- g. Xóa nút đầu của danh sách
- h. Xóa nút kế sau nút được trỏ bởi con trỏ  $q$
- i. Xóa nút có giá trị  $k$

# Hướng dẫn một số hàm

```
#include <stdlib.h>
#include <iostream.h>
#include <alloc.h>
struct node
{
    int    info;
    node  *next;
};
struct list
{
    node  *head,*tail;
};
```

```
int main()
{
    list l;
    list_insert_tail(l);
    list_process(l);
}
```

```
node*      getnode(int x)
{
    node *p=new Node;
    if (p==NULL)
    {
        cout<<"Khong du bo nho";
        exit(1);
    }
    p->info=x;
    p->next=NULL;
    return p;
}
```

```
void    insert_tail(list &l,node *p)
{
    if (l.head==NULL)
        {    l.head=p;
              l.tail=p;
            }
    else
        {
            l.tail->next=p;
            l.tail=p;
        }
}
```

```
void list_insert_tail(list &l)
{
    int x,n;
    cout<<"Danh sach co bao nhieu phan tu : ";cin>>n;
    for (int i=1;i<=n;i++)
    {
        cin>>x;
        insert_tail(l,getnode(x));
    }
}
```

```
void list_process(list l)
{
    node *p = l.head;
    while (p!=NULL)
    {
        cout<<p->info<<" ";
        p=p->next;
    }
}
```

```
void insert_head (list &l,node *p)
{
    if (l.head==NULL)
    {
        l.head=p;
        l.tail=p;
    }
    else
    {
        p->next=l.head;
        l.head=p;
    }
}
```

```
void list_insert_head(list &l)
{
    int x,n;
    cout<<"Nhap vao so n = ";cin>>n;
    for (int i=1;i<=n;i++)
    {
        cin>>x;
        insert_head(l,getnode(x));
    }
}
```

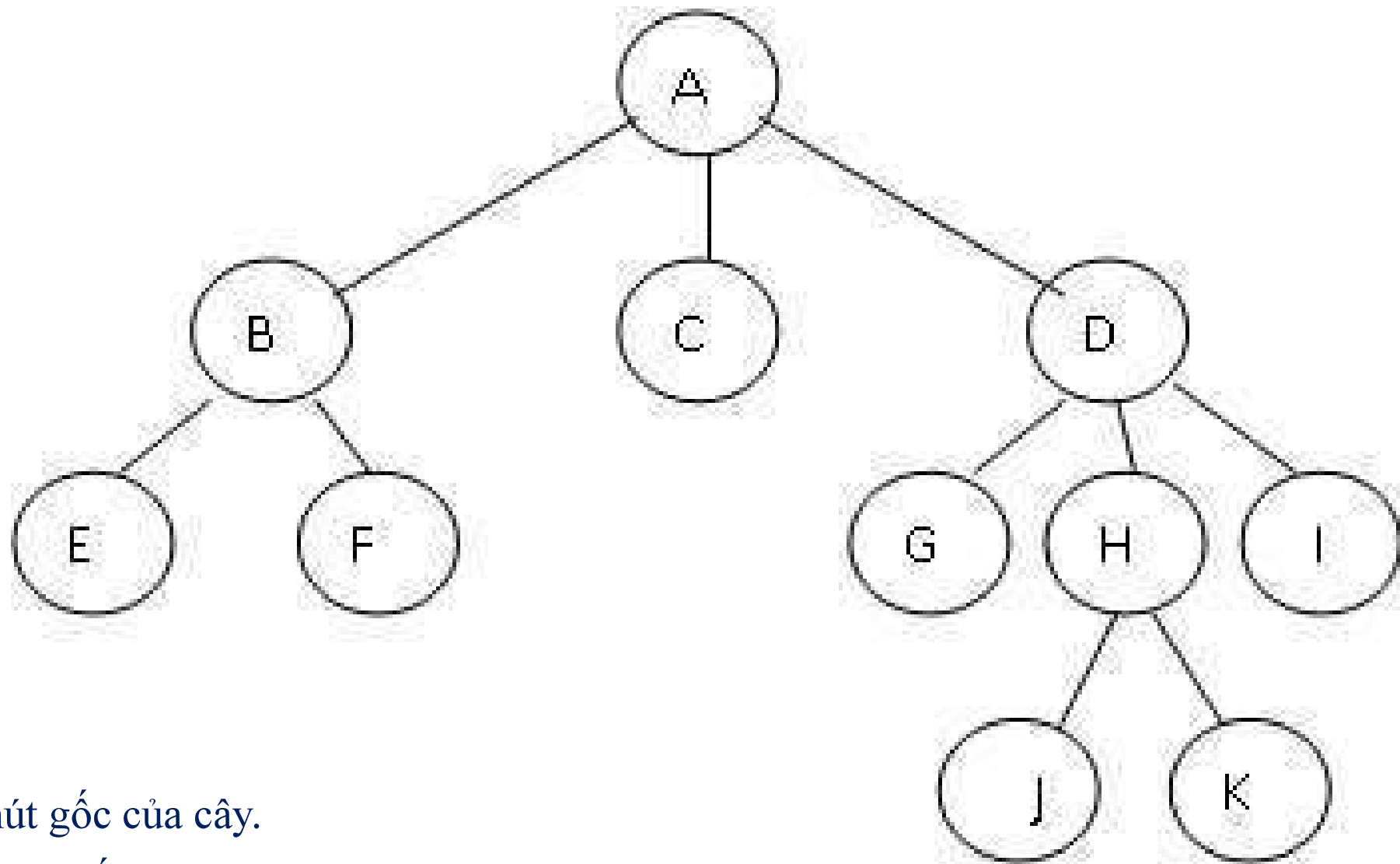
```
void insert_after_nodeq(list &l,node *q,int x)
{
    node *p=taonut(x);
    if (p==NULL) exit(1);
    if (q!=NULL)
        {
            p->next=q->next;
            q->next=p;
            if (q==l.tail)
                l.tail=p;
        }
    else
        insert_head(l,p);
}
```

# Con trỏ với cấu trúc cây

(Hướng dẫn sinh viên nghiên cứu)

# Định nghĩa cấu trúc cây

- Cây là một tập hợp  $T$  các phần tử (gọi là nút của cây) trong đó có một nút đặc biệt được gọi là gốc,
- Các nút còn lại được chia thành những tập rời nhau  $T_1, T_2, \dots, T_n$  theo quan hệ phân cấp trong đó  $T_i$  cũng là một cây.
- Mỗi nút ở cấp  $i$  sẽ quản lý một số nút ở cấp  $i+1$ , đây gọi là quan hệ cha-con.



Nút A gọi là nút gốc của cây.

Các nút B,C,D là gốc của các cây con của A.

Nút A là cha của các nút B,C,D và B,C,D là các nút con của A.

# Một số khái niệm cơ bản

## Bậc của một nút:

- Là số cây con của nút đó.
- Ví dụ bậc của A là 3, bậc của H là 2, bậc của D là 3.

## Bậc của một cây:

- Là bậc lớn nhất của các nút trong cây (số cây con tối đa của một nút thuộc cây). Cây có bậc  $n$  thì gọi là cây  $n$ -phân.
- Ví dụ cây trên có bậc là 3.

## Nút gốc:

- Là nút không có nút cha.
- Ví dụ A là nút gốc của cây.

## Nút lá:

- Là nút có bậc bằng 0.
- Ví dụ E,C,K,I,... là các nút lá

## Nút nhánh:

- Là nút có bậc khác 0 và không phải là gốc .
- Ví dụ B,D,H...là các nút nhánh

## Mức của một nút:

- mức (gốc (T) ) = 0.
- gọi  $T_1, T_2, T_3, \dots, T_n$  là các cây con của  $T_0$
- mức ( $T_1$ ) = mức ( $T_2$ ) = ... = mức ( $T_n$ ) = mức ( $T_0$ ) + 1.
- chẳng hạn nút A có mức là 0, D có mức là 1, G có mức là 2, J có mức là 3.

Độ dài đường đi từ gốc đến nút x:

- Bằng số nút trên đường đi đó trừ đi 1.
- Ví dụ đường đi từ A đến G là 2, đường đi từ A đến K là 3.

Chiều cao của một cây

- Là số mức lớn nhất của nút có trên cây đó cộng thêm 1
- Ví dụ cây trên có chiều cao là 4.

# CÂY NHỊ PHÂN

## Định nghĩa

Cây nhị phân là cây mà mỗi nút có tối đa 2 cây con

Trong thực tế thường gặp các cấu trúc có dạng cây nhị phân.  
Một cây tổng quát có thể biểu diễn thông qua cây nhị phân.

# Một số tính chất của cây nhị phân

Số nút nằm ở mức  $i \leq 2^i$ .

Với  $h$  là chiều cao của cây thì:

Số nút lá  $\leq 2^{h-1}$

thì  $\log_2(\text{số nút trong cây}) \leq h$

Số nút trong cây  $\leq 2^h - 1$ .

# Biểu diễn cây nhị phân T

Cây nhị phân là một cấu trúc bao gồm các phần tử (nút) được kết nối với nhau theo quan hệ “cha-con” với mỗi cha có tối đa 2 con. Để biểu diễn cây nhị phân ta chọn phương pháp cấp phát liên kết. Ứng với một nút, ta dùng một biến động lưu trữ các thông tin:

- Thông tin lưu trữ tại nút.
- Địa chỉ nút gốc của cây con trái trong bộ nhớ.
- Địa chỉ nút gốc của cây con phải trong bộ nhớ.

Khai báo tương ứng trong ngôn ngữ C/C++ có thể như sau:

```
struct node
{
    data info;
    struct node *left,*right;
};
typedef node *tree;
tree root;
```

# Duyệt cây nhị phân

Có 3 kiểu duyệt chính có thể áp dụng trên cây nhị phân:

- Duyệt theo thứ tự trước (NLR),
- Duyệt theo thứ tự giữa (LNR),
- Duyệt theo thứ tự sau (LRN).

Tên của 3 kiểu duyệt này được đặt dựa trên trình tự của việc thăm nút gốc so với việc thăm 2 cây con.

Tương tự như vậy ta có 3 cách duyệt khác là: NRL, RNL, RLN.

# Duyệt theo thứ tự trước (node-left-right)

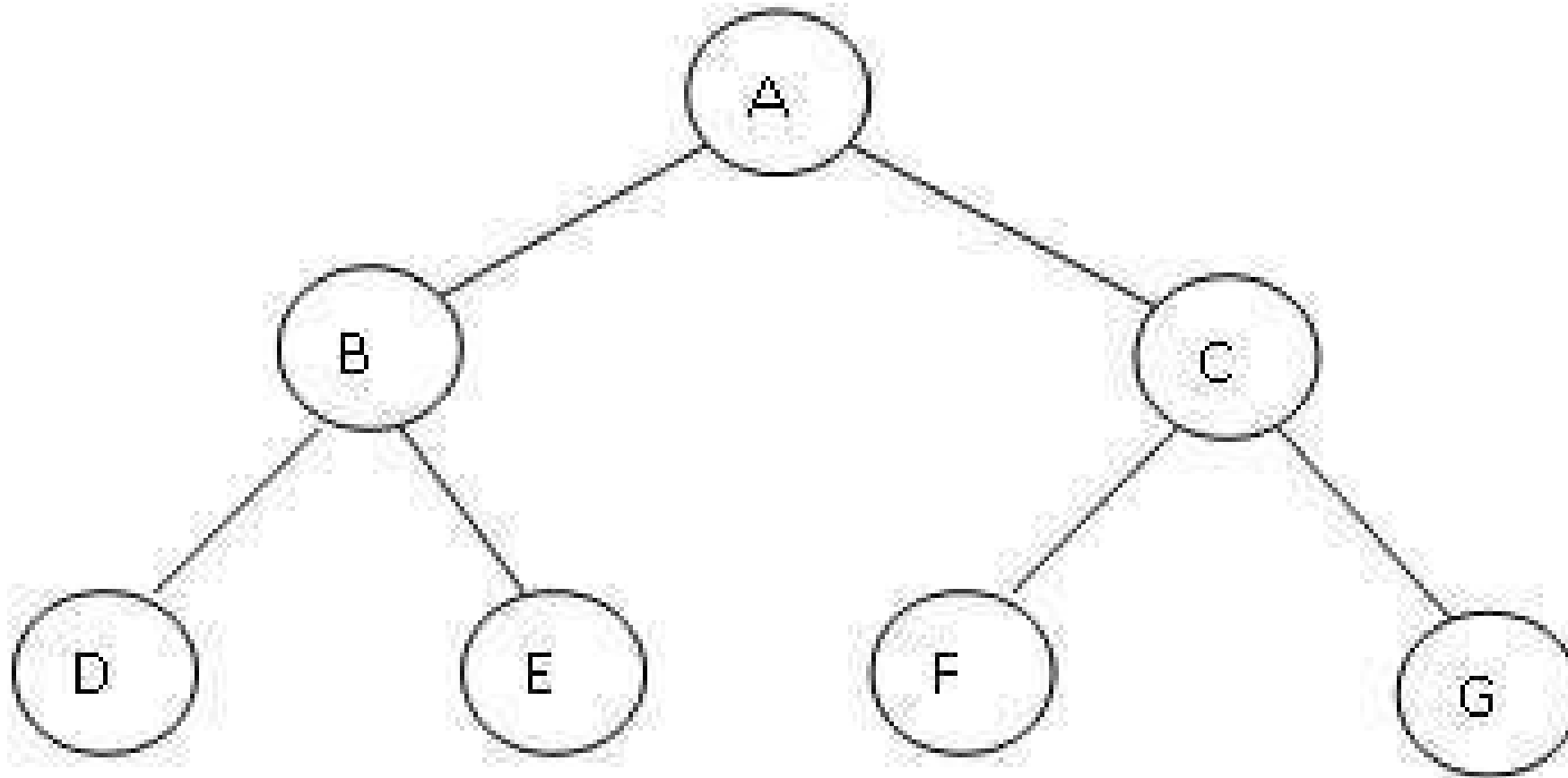
- Kiểu duyệt này trước tiên thăm nút gốc sau đó thăm các nút của cây con trái rồi đến cây con phải. Thủ tục duyệt có thể trình bày đơn giản như sau: (trong mục này việc xử lý các nút chỉ là in giá trị của phần tử đó)

# Duyệt theo thứ tự giữa (Left- Node-Right)

- Kiểu duyệt này trước tiên thăm các nút của cây con trái sau đó thăm nút gốc rồi đến cây con phải. Thủ tục duyệt có thể trình bày đơn giản như sau:

# Duyệt theo thứ tự sau (left-right-node)

- Kiểu duyệt này trước tiên thăm các nút của cây con trái sau đó thăm đến cây con phải rồi cuối cùng mới thăm nút gốc. Thủ tục duyệt có thể trình bày đơn giản như sau:



thì dãy các tên ứng với nút được thăm trong các phép duyệt là:

Theo thứ tự trước: A,B,D,E,C,F,G

Theo thứ tự giữa: D,B,E,A,F,C,G

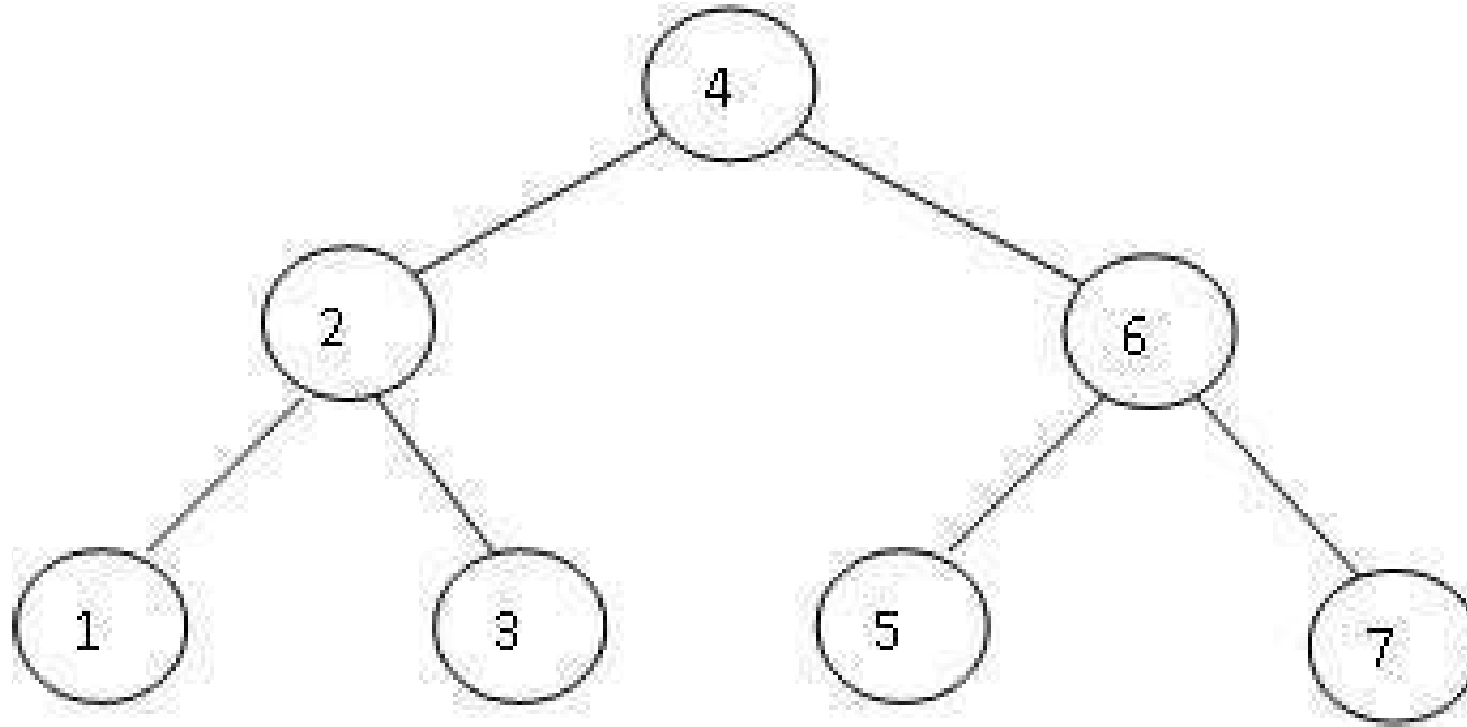
Theo thứ tự sau: D,E,B,F,G,C,A

Bạn đọc hãy liệt kê các nút được thăm ứng với các cách duyệt: NRL, RNL, RLN.

# Cây nhị phân tìm kiếm

## Định nghĩa:

- Cây nhị phân tìm kiếm (CNPTK) là cây nhị phân trong đó tại mỗi nút, khóa của nút đang xét lớn hơn khóa của tất cả các nút thuộc cây con trái và nhỏ hơn khóa của tất cả các nút thuộc cây con phải.
- Nhờ ràng buộc về khóa trên CNPTK, việc tìm kiếm trở nên có định hướng. Hơn nữa, do cấu trúc cây việc tìm kiếm trở nên nhanh đáng kể. Nếu số nút trên cây là  $N$  thì chi phí tìm kiếm trung bình chỉ khoảng  $\log_2 N$ .
- Trong thực tế, khi xét đến cây nhị phân chủ yếu người ta xét CNPTK.



### Duyệt cây

Thao tác duyệt cây trên cây nhị phân tìm kiếm hoàn toàn giống như trên cây nhị phân. Chỉ có một lưu ý nhỏ là khi duyệt theo thứ tự giữa, trình tự các nút duyệt qua sẽ cho ta một dãy các nút theo thứ tự tăng dần của khóa.

chẳng hạn với hình 4.4 thì kết quả của các phép duyệt như sau:

Theo thứ tự trước: 4,2,1,3,6,5,7

Theo thứ tự giữa: 1,2,3,4,5,6,7

Theo thứ tự sau: 1,3,2,5,7,6,4

# Ví dụ

- a. Hãy vẽ cây nhị phân tìm kiếm T biết rằng thứ tự các số nhập vào là: 8, 3, 5, 2, 20, 11, 30, 9, 18, 4.
- b. Hãy duyệt cây T trên theo các thứ tự NLR, LNR, LRN, NRL, RNL, RLN.
- c. Nếu hủy lần lượt các nút 5, 20 (theo thứ tự đó) thì cây sẽ thay đổi như thế nào trong từng bước hủy (vẽ lại cây cho mỗi bước) sao cho cây vẫn là cây nhị phân tìm kiếm.

# VD6. Tạo cây nhị phân tìm kiếm, duyệt cây

```
#include <iostream.h>
```

```
#include <alloc.h>
```

```
struct node
```

```
{
```

```
    int info;
```

```
    struct node *left,*right;
```

```
};
```

```
typedef struct node *tree;
```

```
int    chennutvaoBST(tree &root,int k);  
void   taoBST(tree &root);  
void   LNR(tree root);  
void   NLR(tree root);  
void   LRN(tree root);  
int main()  
{  
    tree root;  
    taoBST(root);  
    LNR(root);  
}
```

# Chèn nút có giá trị $k$ vào cây có gốc là root

- Việc thêm một phần tử  $X$  vào cây phải bảo đảm điều kiện ràng buộc của CNPTK. Ta có thể thêm vào nhiều chỗ khác nhau trên cây, nhưng nếu thêm vào một nút lá sẽ là tiện lợi nhất do ta có thể thực hiện quá trình tương tự thao tác tìm kiếm. Khi chấm dứt quá trình tìm kiếm cũng chính là lúc tìm được chỗ cần thêm.
- Hàm `chennutvaoBST` trả về giá trị  $-1, 0, 1$  khi không đủ bộ nhớ, gập nút cũ hay thành công:

```
int chennutvaoBST(tree &root,int k)
{
    if (root!=NULL)
    {
        if (root->info==k) return 0;
        if (root->info>k)      return chennutvaoBST(root->left,k);
        else    return chennutvaoBST(root->right,k);
    }
    else
    {
        root=new node;
        if (root==NULL) return -1;
        root->info=k;
        root->left=root->right=NULL;
        return 1;
    }
}
```

//-----Tạo cây có gốc là root-----

Ta có thể tạo một cây nhị phân tìm kiếm bằng cách lặp lại quá trình chèn thêm một phần tử vào một cây rỗng.

```
void taoBST(tree &root)
```

```
{
```

```
    int k,n;
```

```
    cout<<"\n\n put n=";cin>>n;
```

```
    root=NULL;
```

```
    for (int i=1;i<=n;i++)
```

```
        {
```

```
            cin>>k;
```

```
            chennutvaoBST(root,k);
```

```
        }
```

```
    root1=root;// có sử dụng cho hàm liên quan đến tìm mức của nút phía sau
```

```
}
```

# Các hàm duyệt cây

```
void LNR(tree root)
{
    if (root!=NULL)
    {
        LNR(root->left);
        cout<<root->info<<" ";
        LNR(root->right);
    }
}
```

```
void NLR(tree root)
{
    if (root!=NULL)
    {
        cout<<root->info<<" ";
        NLR(root->left);
        NLR(root->right);
    }
}
```

```
void LRN(tree root)
{
    if (root!=NULL)
    {
        LRN(root->left);
        LRN(root->right);
        cout<<root->info<<" ";
    }
}
```